

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: XX hrs/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Rigor Técnico	El estudiante aplica este valor al elaborar casos de uso siguiendo una estructura estandarizada (CUN Core, Primera Descomposición, Expandido), documentando cada flujo alterno y excepción sin omitir detalles relevantes para el diseño.
Pensamiento Analítico	Se evidencia al identificar correctamente los actores, sus roles, y las relaciones <code><<include>></code> y <code><<extend>></code> entre casos de uso, representando con precisión la interacción del sistema con el entorno.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Analiza y modela sistemas de software utilizando notación UML (casos de uso, diagramas de clases y secuencia), con el fin de traducir requisitos funcionales en una especificación técnica que guíe la construcción del sistema.
Competencia Específica	El estudiante desarrolla la capacidad de elaborar una especificación completa de casos de uso (CUN Core, Primera Descomposición y Expandido), construir un diagrama de clases conceptual y un diagrama de secuencia a partir de dichos casos de uso, empleando herramientas de modelado como Lucidchart o Draw.io para representar visualmente la solución.

2. Palabras Clave

Caso de Uso del Negocio (CUN): Descripción de una secuencia de interacciones entre uno o más actores y el sistema, que produce un resultado de valor observable para el actor. Se representa en diferentes niveles de detalle: Core, Primera Descomposición y Expandido.

Actor: Entidad externa al sistema (usuario, otro sistema, dispositivo) que interactúa con el sistema para lograr un objetivo. Los actores tienen roles definidos.

<<include>> (Inclusión): Relación donde un caso de uso base incorpora obligatoriamente el comportamiento de otro caso de uso. Se usa para factorizar comportamiento común y reutilizable.

<<extend>> (Extensión): Relación donde un caso de uso base puede opcionalmente extender su comportamiento con otro caso de uso, bajo ciertas condiciones (ej. flujo alternativo o excepción).

CUN Core: El nivel más abstracto del caso de uso, define el objetivo general y el actor principal en una o dos líneas. Representa el "qué" del negocio.

CUN Primera Descomposición: Desglose del caso de uso en pasos de alto nivel (escenario principal), sin detalles técnicos ni condiciones.

CUN Expandido: Especificación detallada que incluye precondiciones, postcondiciones, flujo básico, flujos alternos, excepciones y reglas de negocio.

Diagrama de Clases: Representación estática de la estructura del sistema mostrando clases, atributos, métodos y relaciones (asociación, herencia, composición, agregación).

Diagrama de Secuencia: Representación dinámica que muestra la interacción temporal entre objetos a través del intercambio de mensajes, organizados en líneas de vida (lifelines).

3. Resumen

Este ejercicio demostrativo presenta de forma guiada el proceso completo de especificación y modelado de un sistema a partir de casos de uso. El asistente guiará a los estudiantes a través de:

1. Identificación de actores y elaboración del CUN Core para un sistema de préstamo de libros en biblioteca universitaria.
2. Construcción del CUN en Primera Descomposición (pasos de alto nivel).
3. Desarrollo del CUN Expandido (pre/post condiciones, flujos alternos y excepciones).
4. Aplicación de relaciones **<<include>>** y **<<extend>>** para modularizar el modelo.
5. Transformación del caso de uso en un Diagrama de Clases conceptual.
6. Elaboración de un Diagrama de Secuencia para el flujo principal.
7. Uso de herramientas de modelado ([Draw.io](https://draw.io) / Lucidchart) para representar los diagramas.

La clase debe orientarse a que el estudiante comprenda la trazabilidad desde los requisitos de negocio hasta los modelos estáticos y dinámicos de UML.

4.Contenido

Descripción del ejercicio

Se plantea un escenario real: una biblioteca universitaria desea automatizar su sistema de **préstamo de libros**. El estudiante debe:

- Identificar actores involucrados (bibliotecario, estudiante, sistema de multas, etc.).
- Desarrollar el caso de uso "Realizar préstamo de libro" en sus tres niveles (Core, Primera Descomposición, Expandido).
- Construir un diagrama de clases con las entidades principales (Libro, Usuario, Préstamo, Multa).
- Construir un diagrama de secuencia que ilustre el flujo exitoso del préstamo.
- Utilizar una herramienta de modelado para representar los diagramas (ej. Lucidchart o [Draw.io](https://draw.io)).

Objetivo de la demostración

Demostrar el procedimiento para modelar un sistema desde casos de uso hasta diagramas de clases y secuencia en UML, utilizando relaciones `<<include>>` y `<<extend>>` junto con herramientas de modelado, con el fin de que el estudiante comprenda cómo transformar requisitos funcionales en una especificación técnica visual que sirva como base para la construcción del sistema

Recursos necesarios

- **Software:** Navegador web, editor de texto o procesador de palabras para documentar CUN.
- **Plataformas/Entornos:**
 - [Draw.io](https://draw.io) (diagrams.net) o Lucidchart (cuenta gratuita).
 - Google Docs o Word para plantilla de CUN.
- **Herramientas técnicas:** Plantilla de CUN Expandido (proporcionada por el auxiliar), referencia rápida de notación UML.
- **Archivos de apoyo:** Ejemplo de sistema de biblioteca (descripción breve, actores sugeridos).
- **Equipo necesario:** Computadora con proyector. Opcional: estudiantes replican en sus equipos.

- **Configuración mínima:** Acceso a internet, cuenta gratuita en la herramienta de modelado seleccionada.

Desarrollo paso a paso

Paso 1

Acción inicial: Identificación de actores y elaboración del **CUN Core**.

Objetivo: Establecer el alcance del sistema y los actores involucrados desde una visión de negocio de alto nivel.

Qué observar: El auxiliar presenta el enunciado del sistema de biblioteca y guía una lluvia de ideas para identificar actores:

Actor	Descripción	Rol
Estudiante	Usuario principal, solicita y recibe libros	Primario
Bibliotecario	Registra préstamos y devoluciones, gestiona inventario	Primario
Sistema de Multas	Sistema externo que calcula multas por retraso	Secundario (soporte)

CUN Core elaborado (ejemplo):

- **Nombre:** Realizar préstamo de libro
- **Actor principal:** Estudiante (solicitante) y Bibliotecario (operador)
- **Objetivo:** Que un estudiante pueda llevarse en préstamo un libro disponible, registrando la transacción para control y futura devolución.

Paso 2

Acción: Elaboración del **CUN Primera Descomposición**.

Explicación: Se desglosa el flujo principal en pasos de alto nivel (5-10 pasos), sin detalles técnicos.

Ejemplo para "Realizar préstamo de libro":

Paso	Acción
1	El estudiante entrega el libro y su carné al bibliotecario.
2	El bibliotecario ingresa el ID del libro y la matrícula del estudiante al sistema.
3	El sistema verifica que el libro esté disponible y que el estudiante no tenga multas pendientes ni exceda el límite de libros prestados.
4	El sistema registra el préstamo con fecha actual y fecha de devolución esperada.
5	El sistema actualiza el estado del libro a "Prestado".
6	El bibliotecario entrega el libro al estudiante.

Validación importante: Los pasos deben estar en orden temporal, sin "si" ni condicionales en el flujo principal.

Paso 3

Ejecución principal (parte A): Desarrollo del **CUN Expandido** y relaciones `<<include>>` / `<<extend>>`.

Detalle: El auxiliar completa la plantilla estándar (ver Tabla 1) e introduce:

- **Precondiciones:** El libro debe estar disponible. El estudiante debe estar activo. El bibliotecario autenticado.
- **Postcondiciones exitosas:** Préstamo registrado en el sistema. Libro marcado como prestado.
- **Flujo básico** (del paso 2, expandido con detalles de entrada/salida).
- **Flujos alternos** (condiciones que modifican el flujo):
 - A1: Libro no disponible → mostrar mensaje, fin.
 - A2: Estudiante con multa pendiente → extender con "Calcular y mostrar monto de multa".
 - A3: Estudiante excede límite de préstamos → mostrar mensaje, sugerir devolución.
- **Relaciones:**

- `<<include>>` : "Verificar disponibilidad y estado estudiante" (reutilizable por otros CUN).
- `<<extend>>` : "Consultar multa" se ejecuta solo si existe multa pendiente (condición).

Paso 3 (continuación)

Ejecución principal (parte B): Construcción del **Diagrama de Clases**.

Detalle: A partir del CUN Expandido, se identifican sustantivos y responsabilidades:

Clases identificadas:

- **Libro** (+idLibro, +titulo, +autor, +estado: Disponible/Prestado)
- **Usuario** (+matricula, +nombre, +email)
- **Estudiante** (hereda de Usuario) (+limitePrestamos = 3)
- **Bibliotecario** (hereda de Usuario) (+codigoEmpleado)
- **Prestamo** (+idPrestamo, +fechaPrestamo, +fechaDevolucionEsperada, +fechaDevolucionReal)
- **Multa** (+idMulta, +monto, +pagada)

Relaciones:

- Usuario (abstracta) → Estudiante, Bibliotecario (herencia)
- Prestamo --- Usuario (asociación: 1 usuario realiza N préstamos)
- Prestamo --- Libro (asociación: 1 préstamo tiene 1 libro)
- Prestamo --- Multa (asociación condicional: 0..1)

El auxiliar dibuja el diagrama en [Draw.io](https://draw.io) en vivo, mostrando multiplicidades y navegabilidad.

Paso 3 (continuación)

Ejecución principal (parte C): Construcción del **Diagrama de Secuencia** para el flujo básico.

Detalle: Se modelan los objetos interactuantes y mensajes:

Lifelines: Estudiante, Bibliotecario, InterfazSistema, ControladorPrestamo, Libro, Prestamo

Mensajes (flujo exitoso):

1. Estudiante → Bibliotecario: entrega libro y carné
2. Bibliotecario → InterfazSistema: ingresarDatos(idLibro, matricula)
3. InterfazSistema → ControladorPrestamo: solicitarPrestamo(datos)
4. ControladorPrestamo → Libro: verificarDisponibilidad()
5. Libro → ControladorPrestamo: disponible = true

6. ControladorPrestamo → Usuario (Estudiante): verificarMultas()
7. (alternativo condicional solo si multa)...
8. ControladorPrestamo → Prestamo: crearNuevoPrestamo(...)
9. ControladorPrestamo → Libro: actualizarEstado("Prestado")
10. ControladorPrestamo → InterfazSistema: confirmacionPrestamo()
11. InterfazSistema → Bibliotecario: mostrarExito()
12. Bibliotecario → Estudiante: entrega libro

El auxiliar usa fragmentos **alt** (alternativo) para flujos con multa, **opt** (opcional) para registro de auditoría.

Paso 4

Cierre técnico: Documentación final y verificación de consistencia.

Acciones:

- Verificar que cada paso del flujo básico en el CUN Expandido tenga al menos un mensaje en el diagrama de secuencia correspondiente.
- Confirmar que todas las clases en el diagrama de clases sean necesarias para soportar los casos de uso identificados.
- Revisar que las relaciones **<<include>>** y **<<extend>>** del modelo de casos de uso se reflejen en algún comportamiento modelado (ej. el **<<include>>** "verificar disponibilidad" se ejecuta siempre; el **<<extend>>** "consultar multa" aparece como fragmento **alt**).
- El auxiliar muestra cómo exportar diagramas a PDF/PNG e incrustarlos en un documento de especificación.

Confirmación de aprendizaje: Los estudiantes, de forma individual o en parejas, completan un CUN Expandido para el caso de uso "Devolver libro" en 10 minutos, usando la plantilla provista.

Observaciones importantes

- **Error común:** Confundir **<<include>>** (obligatorio) con **<<extend>>** (condicional). Usar la analogía: include es como un "subprograma" llamado siempre, extend es como un "manejador de excepción" o "interrupción opcional".
- **Error común:** Poner detalles de interfaz de usuario en el CUN Expandido (ej. "el usuario hace clic en botón OK"). El CUN debe describir *qué*, no *cómo*.
- **Buena práctica:** Para el diagrama de clases, comenzar con 3-5 clases principales y añadir complejidad gradualmente.

- **Buena práctica:** El diagrama de secuencia debe tener un nombre de caso de uso como título (ej. "Realizar préstamo - flujo básico").
- **Recomendación de herramienta:** [Draw.io](https://draw.io) tiene plantillas UML nativas. Usar "Arrange → Insert → Template" para comenzar rápido.

Tabla 1 – Plantilla de CUN Expandido (Caso: Realizar préstamo de libro)

Elemento	Descripción
Nombre del CUN	Realizar préstamo de libro
Actor(es)	Estudiante (principal, iniciador), Bibliotecario (secundario, operador), Sistema de Multas (externo)
Precondiciones	El libro existe en el catálogo y no está prestado. El estudiante tiene estado activo en el sistema.
Postcondiciones (éxito)	El préstamo queda registrado con ID único. El libro cambia su estado a "Prestado". La fecha de devolución esperada se fija a +15 días.
Flujo básico (escenario feliz)	1. El estudiante entrega libro y carné al bibliotecario. 2. El bibliotecario ingresa ID del libro y matrícula. 3. El sistema verifica disponibilidad y estado del estudiante (include). 4. El sistema registra el préstamo y genera ID. 5. El sistema actualiza estado del libro. 6. El sistema muestra comprobante. 7. El bibliotecario entrega el libro al estudiante.

Flujos alternos	A1 – Libro no disponible: En paso 3, libro está prestado → sistema muestra mensaje, fin. A2 – Estudiante con multa: En paso 3, estudiante tiene multa pendiente → se ejecuta extensión "Consultar multa", el sistema muestra monto adeudado, se requiere pago antes de continuar. A3 – Excede límite: En paso 3, estudiante tiene 3 préstamos activos → sistema muestra límite alcanzado, fin.
Excepciones	E1 – Fallo en registro de préstamo: Base de datos no disponible → sistema muestra error, reintentar. El libro no cambia de estado.
Reglas de negocio	RN1: El límite de préstamos activos por estudiante es 3. RN2: El plazo estándar de préstamo es 15 días. RN3: La multa diaria por retraso es Q1.00 (aprox USD 0.13).
Suposiciones	El bibliotecario está autenticado en el sistema antes del CUN. No hay problemas de red.
Relaciones	<<include>>: Verificar disponibilidad y estado estudiante <<extend>>: Consultar multa (condición: multa pendiente)

Resultado esperado

Al finalizar el ejercicio, cada estudiante debe ser capaz de producir:

1. **CUN Core, Primera Descomposición y Expandido** documentados para un caso de uso de la biblioteca (entregable escrito).
2. **Diagrama de clases UML** (5-8 clases con relaciones y multiplicidades) utilizando [Draw.io](https://draw.io) o Lucidchart.
3. **Diagrama de secuencia UML** (con 2-3 objetos y fragmentos alt/opt según sea necesario).
4. **Identificar correctamente** en un modelo dado qué relaciones son <<include>> y cuáles <<extend>>, justificando con la condición de obligatoriedad.
5. **Exportar** los diagramas a formato de imagen e incrustarlos en un documento de especificación.

Aplicación práctica

Este ejercicio se conecta directamente con:

- **Ingeniería de Requisitos:** Los casos de uso son el estándar de facto para capturar requisitos funcionales en metodologías tanto robustas como ágiles.
- **Modelado orientado a objetos:** El puente entre análisis y diseño se construye desde casos de uso hacia diagramas de clases y secuencia (método ICONIX, RUP, etc.).
- **Cursos posteriores:** Arquitectura de Software, Desarrollo de Software, Patrones de Diseño, Construcción de Compiladores (análisis de sistemas complejos).
- **Herramientas profesionales:** Lucidchart, [Draw.io](https://draw.io), Visual Paradigm, StarUML, Enterprise Architect son usadas en la industria. Este ejercicio da una base sólida.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Los estudiantes confunden sistemáticamente `<<include>>` con `<<extend>>`, incluso tras varios ejemplos. La condición "si ocurre X opcionalmente → extend (extensión); si ocurre siempre → include" no se fija fácilmente.
- El salto de CUN Expandido (texto) a Diagrama de Secuencia (formal) es grande; muchos estudiantes no identifican qué mensajes modelar ni el orden correcto.
- El diagrama de clases suele salir con relaciones incorrectas (herencia donde debería haber asociación, multiplicidades al revés).
- La herramienta [Draw.io](https://draw.io), aunque potente, tiene una interfaz que abruma a los principiantes (demasiadas opciones, formas, conectores).

Recomendaciones para futuras sesiones:

- **Sesión previa corta (15 min) sobre teoría de relaciones:** usar tarjetas físicas con verbos "siempre" (include) vs "a veces" (extend) y ejemplos cotidianos (ej. "iniciar sesión" es include de casi todo; "recuperar contraseña" es extend al loguearse).
- **Proveer una "plantilla guía" de diagrama de secuencia** con los lifelines ya dibujados y los conectores básicos; que los estudiantes solo completen los mensajes.
- **Usar un sistema de color:** asociaciones (negro), herencia (azul), dependencia (verde punteado) en diagrama de clases para reducir confusión.

- **Mostrar primero herramientas de modelado simples** (draw.io modo offline) y luego avanzar a Lucidchart.
- **Dejar un ejemplo completamente resuelto** en el repositorio del curso, para que los estudiantes rezagados puedan consultarlo.

Qué puntos consideras fue de mayor dificultad para el estudiante:

- **La granularidad de los pasos:** o son demasiado técnicos (detalles de UI) o demasiado abstractos ("el sistema procesa la solicitud" sin desglose).
- **Modelar excepciones y flujos alternos:** tienden a entremezclarlos u olvidar condiciones de disparo.
- **Identificar las operaciones (métodos) en el diagrama de clases:** a menudo listan atributos pero no responsabilidades de comportamiento (ej. "calcularMulta(")).
- **El concepto de "línea de vida" (lifeline) activa vs inactiva** en diagrama de secuencia: confunden el momento en que un objeto se crea o destruye.
- **Herencia mal aplicada:** crean "Usuario" y debajo "Estudiante" y "Bibliotecario", pero ponen atributos redundantes o rompen la generalización.

6. Referencias

1. Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3ª ed.). Addison-Wesley. (Capítulos 4, 6 y 10 sobre casos de uso, diagrama de clases y secuencia).
2. Cockburn, A. (2001). *Writing Effective Use Cases*. Addison-Wesley. (Método de niveles Core, Descomposición, Expandido).
3. Object Management Group (OMG). (2017). *Unified Modeling Language (UML) Specification Version 2.5.1*. Recuperado de <https://www.omg.org/spec/UML/>
4. [Draw.io](https://draw.io) / diagrams.net. (2024). *Plantillas UML*. Recuperado de <https://www.diagrams.net/blog/uml-diagrams>
5. Lucidchart. (2024). *Guía de diagramas de secuencia UML*. Recuperado de <https://www.lucidchart.com/pages/es/diagrama-de-secuencia-uml>
6. Larman, C. (2005). *Applying UML and Patterns* (3ª ed.). Prentice Hall. (Método de análisis a diseño).